

askmydocs

Project Architecture, Data Flow & Design Decisions

Executive Summary

askmydocs is a batch data-catalog pipeline that ingests ~500 YAML data dictionary files (real masked samples plus synthetic neighbors), validates them locally and in Spark, and writes clean Parquet plus a quarantine lane and DQ report. The design prioritizes defensive parsing, explicit schemas, and two-tier data quality (row quarantine vs batch halt). It is the foundation for a future "ask my docs" RAG layer; the chat UI is not built yet.

1. High-Level Architecture

The system is layered: contract -> raw YAML -> validation -> Spark ingest -> Parquet outputs.

```
[Contract]  JSON Schema, domain_topology.md, dq_contract.md
|
[Sources]   data/real/*.yaml + generate_yamls.py -> data/raw/ (~500 files)
|
[Local QA]  validate_real.py, pytest (no Spark)
|
[Spark]     Docker -> spark_session -> ingest_catalog.py
|
[Outputs]   catalog/ | quarantine/ | dq_report/report.json
```

Layers

Layer	Role
Schemas	JSON Schema + domain topology define valid dictionary entries
Data	~500 YAML files mimicking a logistics warehouse data dictionary
Local tools	Generate data, validate real files, pytest on full corpus
Spark jobs	Batch parse, row-level DQ, aggregate contract, write Parquet
Docker	Reproducible local Spark (Java 17 + PySpark 3.5.3)

main.py is a stub; real entry points are scripts, tests, and spark_jobs/.

2. What Each YAML Represents

Each file documents one warehouse table:

- table_name - e.g. pets_tickets_enriched
- overview - purpose, granularity, business_rules
- input_tables - lineage (namespace.table, e.g. data_warehouse.shipments)
- fields - name, description, source path(s), technical_description (SQL-ish)

Domain model (schemas/domain_topology.md)

- 10 business domains: orders, shipments, hubs, fleet, customers, support, billing, inventory, calendar, audit
- Table archetypes: *_enriched, *_events, dim_*/fct_*, stg_*, etc.
- Namespaces: data_warehouse, core_prod_gl, metadata

3. End-to-End Data Flow

Step 1: Build raw corpus (~500 files)

```
data/real/*.yaml --copy-->\
                                +--> data/raw/*.yaml (target: 500)
generate_yamls.py --synthetic-->/
```

generate_yamls.py copies real masked YAMLs, generates synthetic tables to reach 500, uses Faker + domain vocabulary, validates each synthetic doc with jsonschema before write.

Step 2: Local validation (pre-Spark)

- validate_real.py - schema-check real files only
- tests/test_yaml_validity.py - count=500, schema, duplicates, source paths, lineage coverage

Step 3: Production ingest (spark_jobs/ingest_catalog.py)

- 1. Read - binaryFile + *.yaml glob (one row per file)
- 2. Parse - Python UDF -> PARSED_SCHEMA (parse errors captured, not thrown)
- 3. Validate - Spark SQL CASE WHEN -> rejection_reason
- 4. Report - dq_report/report.json
- 5. Halt - DQContractViolation if aggregate thresholds fail
- 6. Write - good -> catalog/, bad -> quarantine/

Sample run: 500 files, 484 good, 16 quarantined (3.2% rejection rate).

Step 4: Run environment

- run.bat -> docker compose run spark
- Container mounts repo at /app, PYTHONPATH=/app
- get_spark(): local[4], 4g driver, UTC, AQE, UI on port 4040
- Production path: GKE + Spark Operator (planned)

4. Ingestion Design: Three Approaches

Script	Approach / status
ingest_v1_text_udf.py	binaryFile + UDF - reliable one-row-per-file
ingest_v2_wholetextfiles.py	RDD wholeTextFiles - schema-inference pitfalls demo
ingest_catalog.py	CANONICAL - v1 parsing + full DQ + Parquet

Chosen pattern: binaryFile + explicit StructType + defensive parsing (coerce input_tables string to list, filter business_rules to strings). Row-level issues go to quarantine; job only fails on aggregate breach.

5. Data Quality: Two-Tier Model

Documented in spark_jobs/dq_contract.md, implemented in ingest_catalog.py.

Per-row (quarantine, job continues)

Parse failure, bad table_name, empty fields, duplicate field names, short purpose/granularity, malformed input_tables.

Aggregate (job fails, no Parquet written)

Rule	Threshold
Volume	file_count < 400
Rejection rate	> 5%
Single-cause dominance	> 50% of rejections from one reason (if >= 5 bad)
Namespace diversity	< 2 distinct input namespaces in good rows

Note: dq_contract.md mentions 8 business domains; implemented check is 2+ input namespaces (data_warehouse vs core_prod_gl). distinct_input_namespaces in reports counts namespace prefixes, not the 10 domains in domain_topology.md.

Explicitly out of scope for ingest

- Whether referenced tables exist (cross-file lineage - Day 4)
- Semantic correctness of descriptions (needs LLM)
- Schema evolution over time (planned: dbt snapshots)

scripts/corrupt_for_dq_testing.py breaks 9 files to test DQ behavior.

6. Schema and Testing Decisions

- JSON Schema is source of truth for generation and pytest
- Spark DQ adds stricter/different rules - intentional split: fast local vs distributed batch
- Synthetic data: weighted domains, cross-domain input_tables, derived fields with SQL-like tech descriptions
- Same pipeline for real + synthetic so downstream dbt/RAG does not special-case source

7. Dependencies and Operations

Tool	Use
uv / pyproject.toml	faker, jsonschema, pyyaml, pytest (Python >= 3.12)
Docker Spark image	pyspark 3.5.3, pyyaml, jsonschema in container
Parquet	Analytics-ready catalog for later search/embedding

8. Roadmap (Implied, Not Built Yet)

- Week 2: dbt snapshots for schema evolution
- Day 4: cross-file lineage integrity
- Week 3 Day 5: GKE Spark Operator
- RAG / ask my docs: retrieve from catalog Parquet using purpose, business_rules, field text

9. Summary

askmydocs is a batch data-catalog pipeline: YAML data dictionaries -> validated, queryable Parquet with quarantine and halt-on-bad-batch semantics. Design emphasizes defensive parsing, explicit schemas, two-tier DQ, Docker-local Spark with a path to Kubernetes, and a 500-file corpus blending real masked docs with synthetic logistics tables for downstream analytics and Q&A.